

PA2-简单复杂的机器：冯诺依曼计算机系统

世界诞生的故事-第二章

先驱已经创造了图灵机。但区区几个数字电路模块搭成的如此简单的机器，又能做些什么呢？先驱说，一切无限的可能，都蕴含其中。

在进行本 PA 前，请在工程目录下执行以下命令进行分支整理，否则将影响你的成绩：

```
git commit --allow-empty -am "before starting pa2"
git checkout master
git merge pa1
git checkout -b pa2
```

不停计算的机器

在 PA1 中，我们已经见识到最简单的计算机 TRM 的工作方式：

```
while (1) {
    从EIP指示的存储器位置取出指令；
    执行指令；
    更新EIP；
}
```

接下来我们就来谈谈这个过程，也就是，CPU 究竟是怎么执行一条指令的。对于大部分指令来说，执行它们都可以抽象成取指-译码-执行的**指令周期**。为了使描述更加清晰，我们借助指令周期中的一些概念来说明指令执行的过程。

取指 (instruction fetch, IF)

取指令要做的事情自然就是将%eip 指向的指令从内存读入到 CPU 中，其实就是一次内存的访问。

译码 (instruction decode, ID)

在取指阶段，计算机拿到了将要执行的指令。让我们也来目睹一下指令的风采，睁大眼睛一看，竟然是个 0 和 1 组成的比特串！

```
10111001 00110100 00010010 00000000 00000000
```

这究竟是什么鬼... 不过想想，计算机也只是个巨大的数字电路，它也只能理解 0 和 1 了。但是，这样的计算机又是如何理解这让人一头雾水的比特串的呢？

让我们先来回想一下指令是做什么的。我们知道 CPU 是用来处理数据的，指令则是用来指示 CPU 具体对什么数据进行什么样的处理。也就是说，我们只要让 CPU 从上面那串神秘的比特串中解读出处理的对象和处理的动作，CPU 就知道我们想让它做什么了。所以相应地，CPU 需要从指令中解读出“操作数”和“操作码”两部分信息。

于是，为了让计算机明白指令的含义，先驱想到了一个办法，那就是你在数字电路课上学习过的查找表！CPU 拿到一条指令之后，可以通过查表的方式得知这条指令的操作数和操作码。这个过程叫译码。

当然，译码逻辑实际上也并非只有一张查找表那么简单，还需要根据不同的指令通过多路选择器选择不同的操作数。回想一下，计算机现在已经有存储器和寄存器了，它们都可以存放操作数，指令中也可以存放立即数。也可能还有二次译码的处理... 不过无论再怎么复杂，我们只需要知道，这个过程终究也只是一些数字电路的事情，毕竟所有需要的信息都在指令里面了，没什么神秘的操作。

执行 (execute, EX)

经过译码之后，CPU 就知道当前指令具体要做什么了，执行阶段就是真正完成指令的工作。现在 TRM 只有加法器这一个执行部件，必要的时候，只需要往加法器输入两个源操作数，就能得到执行的结果了。之后还要把结果写回到目的操作数中，可能是寄存器，也可能是内存。

更新%eip

执行完一条指令之后，CPU 就要执行下一条指令。在这之前，CPU 需要更新%eip 的值，让%eip 加上刚才执行完的指令的长度，即可指向下一条指令的位置。

于是, 计算机不断地重复上述四个步骤, 不断地执行指令, 直到永远.

也许你会疑惑, 这个只能做加法的 TRM, 究竟还能做些什么呢? 对于采用补码表示的计算机, 能做加法自然就能做减法. 如果再添加一条条件跳转指令 `jne r, addr`: 当寄存器 `r` 不为 0 时, `%eip` 跳转到 `addr` 处, TRM 就大不一样了. 科学家证明了, 只要有 `inc`, `dec`, `jne` 这三条指令, 就可以实现"所有"的算法! (这里的"所有"是指可计算理论中的"所有可计算的算法")也就是说, 现代计算机可以解决的纯粹的计算问题, 这个只有三条指令的 TRM 也可以解决. 例如通过 `jne` 和 `dec` 的组合可以实现循环, 循环执行 `inc` 可以实现任意数的加法, 循环执行加法可以实现乘法... 甚至科学家还证明了, 仅仅通过这三条指令, 就可以编写一个和 NEMU 功能等价的程序! 这下可不得了了, 没想到这个弱不禁风的 TRM 竟然深藏着擎天撼地的威力! 不过, 虽然这个只有三条指令的 TRM 可以解决所有可计算的问题, 但却低效得让人无法忍受. 为此, 先驱决定往 TRM 中加入更多高效的指令.

RTFM

我们在上一小节中已经在概念上介绍了一条指令具体如何执行, 其中有的概念甚至显而易见得难以展开. 不过 x86 这一庞然大物背负着太多历史的包袱, 但当我们决定往 TRM 中添加各种高效的 x86 指令时, 也同时意味着我们无法回避这些繁琐的细节. 首先你需要了解指令确切的行为, 为此, 你需要阅读 i386 手册中指令集相关的章节. 并且上网检索学习 X86 指令格式的内容.

RISC 与 CISC 平行的另一个世界

你是否觉得 x86 指令集的格式特别复杂? 这其实是 CISC 的一个特性, 不惜使用复杂的指令格式, 牺牲硬件的开发成本, 也要使得一条指令可以多做事情, 从而提高代码的密度, 减小程序的大小. 随着时代的发展, 架构师发现 CISC 中复杂的控制逻辑不利于提高处理器的性能, 于是 RISC 应运而生. RISC 的宗旨就是简单, 指令少, 指令长度固定, 指令格式统一, 这和 KISS 法则有异曲同工之妙. [这里](#)有一篇对比 RISC 和 CISC 的小短文.

另外值得推荐的是[这篇文章](#), 里面讲述了一个从 RISC 世界诞生, 到与 CISC 世界融为一体的故事, 体会一下 RISC 的诞生对计算机体系结构发展的里程碑意义.

RTFSC(2)

下面我们来介绍 NEMU 的框架代码是如何执行指令的.

数据结构

首先先对这个过程中的两个重要的数据结构进行说明.

- ❖ `nemu/src/cpu/exec/exec.c` 中的 `opcode_table` 数组. 这就是我们之前提到的译码查找表了, 这一张表通过操作码 `opcode` 来索引, 每一个 `opcode` 对应相应指令的译码函数, 执行函数, 以及操作数宽度.
- ❖ `nemu/src/cpu/decode/decode.c` 中的 `decoding` 结构. 它用于记录一些全局译码信息供后续使用, 包括操作数的类型, 宽度, 值等信息. 其中的 `src` 成员, `src2` 成员和 `dest` 成员分别代表两个源操作数和一个目的操作数. `nemu/include/cpu/decode.h` 中定义了三个宏 `id_src`, `id_src2` 和 `id_dest`, 用于方便地访问它们.

执行流程

然后对 `exec_wrapper()` 的执行过程进行简单介绍.

- ❖ 首先将当前的 `%eip` 保存到全局译码信息 `decoding` 的成员 `seq_eip` 中, 然后将其地址被作为参数送进 `exec_real()` 函数中. `seq` 代表顺序的意思, 当代码从 `exec_real()` 返回时, `decoding.seq_eip` 将会指向下一条指令的地址. `exec_real()` 函数通过宏 `make_EHelper` 来定义:

```
#define make_EHelper(name) void concat(exec_, name) (vaddr_t *eip)
```

其含义是"定义一个执行阶段相关的 helper 函数", 这些函数都带有一个参数 `eip`. NEMU 通过不同的 helper 函数来模拟不同的步骤.

在 `exec_real()` 中:

- ❖ 首先通过 `instr_fetch()` 函数(在 `nemu/include/cpu/exec.h` 中定义)进行取指, 得到指令的第一个字节, 将其解释成 `opcode` 并记录在全局译码信息 `decoding` 中.
- ❖ 根据 `opcode` 查阅译码查找表, 得到操作数的宽度信息, 并通过调用 `set_width()` 函数将其记录在全局译码信息 `decoding` 中.
- ❖ 调用 `idex()` 对指令进行进一步的译码和执行

`idex()` 函数会调用译码查找表中的相应的译码函数进行操作数的译码. 译码函数统一通过宏 `make_DHelper`

来定义(在 `nemu/src/cpu/decode/decode.c` 中):

```
#define make_DHelper(name) void concat(decode_, name) (vaddr_t *eip)
```

它们的名字主要采用 i386 手册附录 A 中的操作数表示记号, 例如 `I2r` 表示将立即数移入寄存器, 其中 `I` 表示立即数, `2` 表示英文 `to`, `r` 表示通用寄存器, 更多的记号请参考 i386 手册. 译码函数会把指令中的操作数信息分别记录在全局译码信息 `decoding` 中

这些译码函数会进一步分解成各种不同操作数的译码的组合, 以实现操作数译码的解耦. 操作数译码函数统一通过宏 `make_DopHelper` 来定义(在 `nemu/src/cpu/decode/decode.c` 中, `decode_op_rm()` 除外):

```
#define make_DopHelper(name) void concat(decode_op_, name) (vaddr_t *eip, Operand *op, bool load_val)
```

它们的名字主要采用 i386 手册附录 A 中的操作数表示记号. 操作数译码函数会把操作数的信息记录在结构体 `op` 中, 如果操作数在指令中, 就会通过 `instr_fetch()` 将它们从 `eip` 所指向的内存位置取出. 为了使操作数译码函数更易于复用, 函数中的 `load_val` 参数会控制是否需要将该操作数读出到全局译码信息 `decoding` 供后续使用. 例如如果一个内存操作数是源操作数, 就需要将这个操作数从内存中读出来供后续执行阶段来使用; 如果它仅仅是一个目的操作数, 就不需要从内存读出它的值了, 因为执行这条指令并不需要这个值, 而是将新数据写入相应的内存位置.

`idex()` 函数中的译码过程结束之后, 会调用译码查找表中的相应的执行函数来进行真正的执行操作. 执行函数统一通过宏 `make_EHelper` 来定义, 它们的名字是指令操作本身. 执行函数通过 RTL 来描述指令真正的执行功能(RTL 将在下文介绍). 其中 `operand_write()` 函数(在 `nemu/src/cpu/decode/decode.c` 中定义)会根据第一个参数中记录的类型的不同进行相应的写操作, 包括写寄存器和写内存.

从 `idex()` 返回后, `exec_real()` 最后会通过 `update_eip()` 对 `%eip` 进行更新.

上文已经把一条指令在 NEMU 中执行的流程进行了大概的介绍. 如果觉得上文的内容不易理解, 可以结合 X86 指令系统 PPT 来 RTFSC. 阅读的时候需要一定的耐心.

立即数背后的故事

在 `decode_op_l()` 函数中通过 `instr_fetch()` 函数获得指令中的立即数. 别看这里就这么一行代码, 其实背后隐藏着针对字节序的慎重考虑. 我们知道 x86 是小端机, 当你使用高级语言或者汇编语言写了一个 32 位常数 `0x1234` 的时候, 在生成的二进制代码中, 这个常数对应的字节序列如下(假设这个常数在内存中的起始地址是 `x`):

```
x    x+1  x+2  x+3
+----+----+----+----+
| 34 | 12 | 00 | 00 |
+----+----+----+----+
```

而大多数 PC 机都是小端架构(我们相信没有同学会使用 IBM 大型机来做 PA), 当 NEMU 运行的时候, `op_src->imm=instr_fetch(eip, 4)`, 这行代码会将 `34 12 00 00` 这个字节序列原封不动地从内存读入 `imm` 变量中, 主机的 CPU 会按照小端方式来解释这一字节序列, 于是会得到 `0x1234`, 符合我们的预期结果.

Motorola 68k 系列的处理器都是大端架构的. 现在问题来了, 考虑以下两种情况:

- ❖ 假设我们需要将 NEMU 运行在 Motorola 68k 的机器上(把 NEMU 的源代码编译成 Motorola 68k 的机器码)
- ❖ 假设我们需要编写一个新的模拟器 NEMU-Motorola-68k, 模拟器本身运行在 x86 架构中, 但它模拟的是 Motorola 68k 程序的执行

在这两种情况下, 你需要注意些什么问题? 为什么会产生这些问题? 怎么解决它们? 事实上不仅仅是立即数的访问, 长度大于 1 字节的内存访问都需要考虑类似的问题. 我们在这里把问题统一抛出来, 以后就不再单独讨论了.

结构化程序设计

细心的你会发现以下规律:

- ❖ 对于同一条指令的不同形式, 它们的执行阶段是相同的. 例如 `add_I2E` 和 `add_E2G` 等, 它们的执行阶段都是把两个操作数相加, 把结果存入目的操作数.
- ❖ 对于不同指令的同一种形式, 它们的译码阶段是相同的. 例如 `add_I2E` 和 `sub_I2E` 等, 它们的译码阶段都是识别出一个立即数和一个 E 操作数.
- ❖ 对于同一条指令同一种形式的不同操作数宽度, 它们的译码阶段和执行阶段都是非常类似的. 例如 `add_I2E_b`, `add_I2E_w` 和 `add_I2E_l`, 它们都是识别出一个立即数和一个 E 操作数, 然后把相加的结果存入 E 操作数.

这意味着,如果独立实现每条指令不同形式不同操作数宽度的 `helper` 函数,将会引入大量重复的代码.需要修改的时候,相关的所有 `helper` 函数都要分别修改,遗漏了某一处就会造成 `bug`,工程维护的难度急速上升.一种好的做法是把译码,执行和操作数宽度的相关代码分离开来,实现解耦,也就是在程序设计课上提到的结构化程序设计.

在框架代码中,实现译码和执行之间的解耦的是 `idex()` 函数,它依次调用 `opcode_table` 表项中的译码和执行的 `helper` 函数,这样我们就可以分别编写译码和执行的 `helper` 函数了.实现操作数宽度和译码,执行这两者之间的解耦的是 `id_src`, `id_src2` 和 `id_dest` 中的 `width` 成员,它们记录了操作数宽度,译码和执行的过程中会根据它们进行不同的操作,通过同一份译码函数和执行函数实现不同操作数宽度的功能.

为了易于使用,框架代码中使用了一些宏,我们在这里把相关的宏整理出来,供大家参考.

宏	含义
<code>nemu/include/macro.h</code>	
<code>str(x)</code>	字符串 "x"
<code>concat(x, y)</code>	<code>tokenxy</code>
<code>nemu/include/cpu/reg.h</code>	
<code>reg_l(index)</code>	编码为 <code>index</code> 的 32 位 GPR
<code>reg_w(index)</code>	编码为 <code>index</code> 的 16 位 GPR
<code>reg_b(index)</code>	编码为 <code>index</code> 的 8 位 GPR
<code>nemu/include/cpu/decode.h</code>	
<code>id_src</code>	全局变量 <code>decoding</code> 中源操作数成员的地址
<code>id_src2</code>	全局变量 <code>decoding</code> 中 2 号源操作数成员的地址
<code>id_dest</code>	全局变量 <code>decoding</code> 中目的操作数成员的地址
<code>make_Dhelper(name)</code>	名为 <code>decode_name</code> 的译码函数的原型说明
<code>nemu/src/cpu/decode.c</code>	
<code>make_Dophelper(name)</code>	名为 <code>decode_op_name</code> 的操作数译码函数的原型说明
<code>nemu/include/cpu/exec.h</code>	
<code>make_Ehelper(name)</code>	名为 <code>exec_name</code> 的执行函数的原型说明
<code>print_asm(...)</code>	将反汇编结果的字符串打印到缓冲区 <code>decoding.assembly</code> 中
<code>suffix_char(width)</code>	操作数宽度 <code>width</code> 对应的后缀字符
<code>print_asm_template1(instr)</code>	打印单目操作数指令 <code>instr</code> 的反汇编结果
<code>print_asm_template2(instr)</code>	打印双目操作数指令 <code>instr</code> 的反汇编结果
<code>print_asm_template3(instr)</code>	打印三目操作数指令 <code>instr</code> 的反汇编结果

强大的宏

如果你知道 C++ 的 "模板" 功能,你可能会建议使用它,但事实上在这里做不到.我们知道宏是在编译预处理阶段进行处理的,这意味着宏的功能不受编译阶段的约束(包括词法分析,语法分析,语义分析);而 C++ 的模板是在编译阶段进行处理的,这说明它会受到编译阶段的限制.理论上来说,必定有一些事情是宏能做到,但 C++ 模板做不到.一个例子就是框架代码中的拼接宏 `concat()`,它可以把两个 `token` 连接成一个新的 `token`;而在 C++ 模板进行处理的时候,词法分析阶段已经结束了,因而不可能通过 C++ 模板生成新的 `token`.

计算机世界处处都是 `tradeoff`,有好处自然需要付出代价.由于处理宏的时候不会进行语法检查,因为宏而造成的错误很有可能不会马上暴露.例如以下代码:

```
#define N 10;
int a[N];
```

在编译的时候,编译器会提示代码的第 2 行有语法错误.但如果你光看第 2 行代码,你很难发现错误,甚至会怀疑编译器有 `bug`.那宏到底要不要用呢?一种客观的观点是,在你可以控制的范围内使用.这就像 `goto` 语句一样,

当你希望在多重循环中从最内层循环直接跳出所有循环, goto 是最方便的做法. 但如果代码中到处都是 goto, 已经严重影响到代码段的可读性了, 这种情况当然是不可取的.

用 RTL 表示指令行为

NEMU 使用 RTL (寄存器传输语言) 来描述 x86 指令的行为. 这样做的好处是可以提高代码的复用率, 使得指令模拟的实现更加规整. 同时 RTL 也可以作为一种 IR (中间表示) 语言, 将来可以很方便地引入即时编译技术对 NEMU 进行优化, 即使你在 PA 中不一定有机会感受到这一好处.

下面我们对 NEMU 中使用的 RTL 进行一些说明, 首先是 RTL 寄存器的定义. RTL 寄存器是 RTL 指令专门使用的寄存器. 在 NEMU 中, RTL 寄存器统一使用 `rtlreg_t` 来定义, 而 `rtlreg_t` (在 `nemu/include/common.h` 中定义) 其实只是一个 `uint32_t` 类型:

```
typedef uint32_t rtlreg_t;
```

在 NEMU 中, RTL 寄存器只有以下这些:

- ❖ x86 的八个通用寄存器 (在 `nemu/include/cpu/reg.h` 中定义)
- ❖ `id_src`, `id_src2` 和 `id_dest` 中的访存地址 `addr` 和操作数内容 `val` (在 `nemu/include/cpu/decode.h` 中定义). 从概念上看, 它们分别与 MAR 和 MDR 有异曲同工之妙
- ❖ 临时寄存器 `t0~t3` (在 `nemu/src/cpu/decode/decode.c` 中定义)
- ❖ 0 寄存器 `tzero` (在 `nemu/src/cpu/decode/decode.c` 中定义), 它只能读出 0, 不能写入

有了 RTL 寄存器, 我们就可以定义 RTL 指令对进行的操作了. 在 NEMU 中, RTL 指令有两种 (在 `nemu/include/cpu/rtl.h` 中定义). 一种是 RTL 基本指令, 它们的特点是在即时编译技术里面可以只使用一条机器指令来实现相应的功能, 同时也不需要使用临时寄存器, 可以看做是最基本的 x86 指令中的最基本的操作. RTL 基本指令包括:

- ❖ 立即数读入 `rtl_li`
- ❖ 算术运算和逻辑运算, 包括寄存器-寄存器类型 `rtl_(add|sub|and|or|xor|shl|shr|sar|slt|sltu)` 和立即数-寄存器类型 `rtl_(add|sub|and|or|xor|shl|shr|sar|slt|sltu) i`
- ❖ 内存的访存 `rtl_lm` 和 `rtl_sm`
- ❖ 通用寄存器的访问 `rtl_lr_(b|w|l)` 和 `rtl_sr_(b|w|l)`

第二种 RTL 指令是 RTL 伪指令, 它们是通过 RTL 基本指令或者已经实现的 RTL 伪指令来实现的, 包括:

- ❖ 带宽度的通用寄存器访问 `rtl_lr` 和 `rtl_sr`
- ❖ EFLAGS 标志位的读写 `rtl_set_(CF|OF|ZF|SF|IF)` 和 `rtl_get_(CF|OF|ZF|SF|IF)`
- ❖ 其它常用功能, 如数据移动 `rtl_mv`, 符号扩展 `rtl_sext` 等

其中大部分 RTL 伪指令还没有实现, 必要的时候你需要实现它们. 有了这些 RTL 指令之后, 我们就可以方便地通过若干条 RTL 指令来实现每一条 x86 指令的行为了.

实现新指令

对译码, 执行和操作数宽度的解耦实现以及 RTL 的引入对 NEMU 中实现一条新的 x86 指令提供了很大的便利, 为了实现一条新指令, 你只需要:

- ❖ 在 `opcode_table` 中填写正确的译码函数, 执行函数以及操作数宽度
 - ❖ 用 RTL 实现正确的执行函数, 需要注意使用 RTL 伪指令时不要把临时变量中有意义的值覆盖了
- 框架代码把绝大部分译码函数和执行函数都定义好了, 你可以很方便地使用它们.

如果你读过上文的扩展阅读材料中关于 RISC 与 CISC 融为一体的故事, 你也许会记得 CISC 风格的 x86 指令最终被分解成 RISC 风格的微指令在计算机中运行, 才让 x86 在这场旷日持久的性能大战中得以存活下来的故事. 如今 NEMU 在经历了第二次重构之后, 也终于引入了 RISC 风格的 RTL 来实现 x86 指令, 这也许是冥冥之中的安排吧.

运行第一个 C 程序

你在 PA2 的第一个任务,就是实现若干条指令,使得第一个简单的 C 程序可以在 NEMU 中运行起来. 这个简单的 C 程序的代码是 `nexus-am/tests/cputest/tests/dummy.c`, 它什么都不做就直接返回. 在 `nexus-am/tests/cputest` 目录下键入 `make ARCH=x86-nemu ALL=dummy run` 编译 dummy 程序, 并启动 NEMU 运行它.

事实上,并不是每一个程序都可以在 NEMU 中运行,nexus-am/子项目专门用于编译出能在 NEMU 中运行的程序,我们在下一小节中会再来介绍它.

在 NEMU 中运行 dummy 程序,你会发现 NEMU 输出以下信息:

```

invalid opcode(eip = 0x0010000a): e8 01 00 00 00 90 55 89 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x0010000a is not implemented.
2. Something is implemented incorrectly.

Find this eip value(0x0010000a) in the disassembling result to distinguish which case
it is.

If it is the first case, see

      _      _      _      _      _
( )_ \ / _ \ / /   | \ / |
-  _ ) | ( ) / / _   | \ / | _ _ _ _ _ _ _ _ _ |
| | | < > _ < | ' _ \   | | \ | / _ | ' _ \ | | | / _ | | | | | | | |
| | _ ) | ( ) | ( ) | | | | | ( | | | | | | | ( | | |
| | _ _ / \ _ / \ _ /   | |   | | \ _ , - | | | | \ _ , - | | \ _ , - | |

for more details.

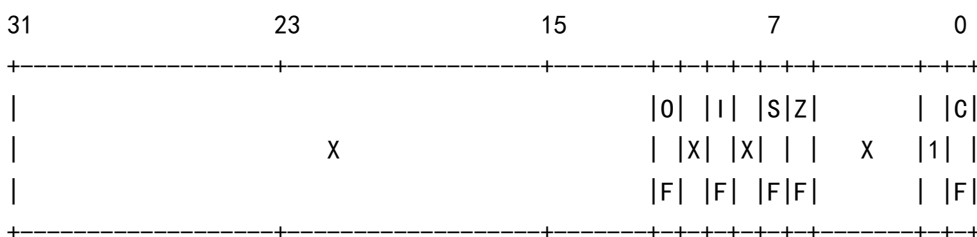
If it is the second case, remember:
* The machine is always right!
* Every line of untested code is always wrong!

```

这是因为你还没有实现以 0xe8 为首字节的指令, 因此, 你需要开始在 NEMU 中添加指令了. 要实现哪些指令才能让 dummy 在 NEMU 中运行起来呢? 答案就在其反汇编结果 (nexus-am/tests/cputest/build/dummy-x86-nemu.txt) 中. 查看反汇编结果, 你发现只需要添加 call, push, sub, xor, pop, ret 六条指令就可以了. 每一条指令还有不同的形式, 根据 KISS 法则, 你可以先实现只在 dummy 中出现的指令形式. 通过指令的 opcode 可以确定具体的形式.

这里要再次强调,你务必通过 i386 手册来查阅指令的功能,不能想当然.手册中给出了指令功能的完整描述(包括做什么事,怎么做的,有什么影响),一定要仔细阅读其中的每一个单词,对指令功能理解错误和遗漏都会给以后的调试带来巨大的麻烦.

- ❖ `call:call` 指令有很多形式, 不过在 PA 中只会用到其中的几种, 现在只需要实现 `CALL rel32` 的形式就可以了。`%eip` 的跳转可以通过将 `decoding.is_jump` 设为 1, 并将 `decoding.jump_eip` 设为跳转目标地址来实现, 这时在 `update_eip()` 函数中会把跳转目标地址作为新的 `%eip`, 而不是顺序意义下的下一条指令的地址
- ❖ `push, pop`: 现在只需要实现 `PUSH r32` 和 `POP r32` 的形式就可以了, 它们可以很容易地通过 `rtl_push` 和 `rtl_pop` 来实现
- ❖ `sub`: 在实现 `sub` 指令之前, 你首先需实现 `EFLAGS` 寄存器. 你只需要在寄存器结构体中添加 `EFLAGS` 寄存器即可. `EFLAGS` 是一个 32 位寄存器, 它的结构如下:



在 NEMU 中,我们只会用到 EFLAGS 中以下的 5 个位:CF, ZF, SF, IF, OF, 标记成 X 的位不必关心,它们的功能可暂不实现.关于 EFLAGS 中每一位的含义,请查阅 i386 手册.添加 EFLAGS 寄存器需要用到结构体的位域(bit field)功能,如果你从未听说过位域,请查阅相关资料.关于 EFLAG 的初值,我们遵循 i386 手册中提到的约定,你需要在 i386 手册的第 10 章中找到这一初值,然后在 restart() 函数中对 EFLAGS 寄存器进行初始化.实现了 EFLAGS 寄存器之后,再实现相关的 RTL 指令,之后你可以通过这些 RTL 指令来实现 sub 指令了

- ❖ xor, ret:RTFM 吧

运行第一个客户程序

在 NEMU 中通过 RTL 指令实现上文提到的指令, 具体细节请务必参考 i386 手册. 实现成功后, 在 NEMU 中运行客户程序 dummy, 你将会看到 HIT GOOD TRAP 的信息.

温馨提示

PA2 阶段 1 到此结束.

程序, 运行时环境与 AM

现代指令系统

我们已经成功在 TRM 上运行 dummy 程序了, 然而这个程序什么都没做就结束了, 一点也不过瘾啊. 为了让 NEMU 支持大部分程序的运行, 你还需要实现更多的指令:

- ❖ Data Movement Instructions: `mov`, `push`, `pop`, `leave`, `cld`(在 i386 手册中为 `cdq`), `movsx`, `movzx`
- ❖ Binary Arithmetic Instructions: `add`, `inc`, `sub`, `dec`, `cmp`, `neg`, `adc`, `sbb`, `mul`, `imul`, `div`, `idiv`
- ❖ Logical Instructions: `not`, `and`, `or`, `xor`, `sal`(`shl`), `shr`, `sar`, `setcc`, `test`
- ❖ Control Transfer Instructions: `jmp`, `jcc`, `call`, `ret`
- ❖ Miscellaneous Instructions: `lea`, `nop`

框架代码已经实现了上述红色标记的指令, 但并没有填写 `opcode_table`. 此外, 某些需要更新 EFLAGS 的指令并没有完全实现好(框架代码中已经插入了 `TODO()` 作为提示), 你还需要编写相应的功能.

运行时环境与 AM

但并不是有了足够的指令就能运行更多的程序. 我们之前提到"并不是每一个程序都可以在 NEMU 中运行", 现在我们来解释一下背后的缘由.

从直觉上来看, 让 TRM 来支撑一个功能齐全的操作系统运行还是比较勉强的. 这给我们的感觉就是, 计算机也有一定的"功能强弱"之分, 计算机越"强大", 就能跑越复杂的程序. 换句话说, 程序的运行其实是对计算机的功能有需求的. 在你运行 Hello World 程序时, 你敲入一条命令(或者点击一下鼠标), 程序就成功运行了, 但这背后其实隐藏着操作系统开发者和库函数开发者的无数汗水. 一个事实是, 应用程序的运行都需要运行时环境的支持, 包括加载, 销毁程序, 以及提供程序运行时的各种动态链接库(你经常使用的库函数就是运行时环境提供的)等. 为了让客户程序在 NEMU 中运行, 现在轮到你来提供相应的运行时环境的支持了. 不用担心, 由于 NEMU 目前的功能并不完善, 我们必定无法向用户程序提供 GNU/Linux 般的运行时环境.

我们先来讨论一下程序执行究竟需要些什么.

- ❖ 程序需要有地方存放代码和数据, 于是需要内存
- ❖ 程序需要执行, 于是需要 CPU 以及指令集
- ❖ 对于需要运行结束的程序, 需要有一种结束运行的方法

事实上, 可以在 TRM 上运行的程序都对计算机有类似的需求. 我们把这些计算机相关的需求抽象成统一的 API 提供给程序, 这样程序就不需要关心计算机硬件相关的细节了. 这正是 AM (Abstract machine) 项目的意义所在.

什么是 AM?

你或许会觉得 NEMU 与 AM 的关系有点模糊不清, 让我们还是来看 ATM 机的例子.

说起 ATM 机, 你脑海里一定会想起一个可以存款, 取款, 查询余额, 转账的机器. 我们不妨把你脑海里的这个机器的模型称为抽象 ATM 机. 从用户的角度来说, 用户对 ATM 机的功能是有期望的: 要能存款, 取款, 查询余额, 转账.

从银行的角度来说, 不同银行的 ATM 机千差万别: 存款的加密方式, 交易时使用的自定义通信协议, 余额在银行系统里面的表示和组织方式... 不同银行的 ATM 机之间存在这么多细节上的差异, 怎么样才能让用户方便地使用 ATM 机呢? 那就是, 为不同银行的 ATM 机分别实现上文提到的抽象 ATM 机的功能: 只要 ATM 机实现了存款, 取款和查询余额的这组统一的功能, 和用户对抽象 ATM 机的认识匹配上, 用户就可以方便地使用这台 ATM 机, 而不必关心 ATM 机的上述细节.

在 NEMU 和 AM 的关系中, 程序就像是用户, AM 就像是抽象 ATM 机, 我们实现 NEMU 这个计算机就像是造一台新的(虚拟的)ATM 机, 也就像我们在 PA1 中提到的, 写一个支付宝 APP. 同样的道理, 程序对计算机的功能是有期望的: 要能计算, 输入输出... 这些功能的期望组成了一台抽象计算机 AM, 它刻画了一台真实计算机应该具备的功能. 但不同计算机的硬件配置各不相同, ISA 也千差万别, 怎么样才能让程序方便地运行呢? 那就是, 为不同的计算机分别实现 AM 的功能: 只要计算机实现了 AM 定义的一组统一的 API, 就能和程序对计算机的功能期望匹配上, 程序就可以方便地在计算机上运行, 而不必关心计算机的底层细节.

有兴趣折腾的同学还可以来理解一下真机, NEMU 和 AM 这三者的关系. 我们会发现, 无论是真实的 ATM 机还是支付宝 APP, 都符合我们对的抽象 ATM 机的认知: 它们都能存款, 取款, 查询余额, 转账. 也正因为如此, 支付宝 APP

刚推出的时候,我们才能很容易上手:虽然支付宝 APP 是个虚拟的 ATM 机,但我们还是可以很容易根据我们对抽象 ATM 机的认知来使用它。

回到 NEMU 的例子中来,我们还是用 ATM 机的例子来比喻:真机就像是一台真实的 ATM 机, NEMU 这个虚拟机就像是一个支付宝 APP, AM 还是我们概念上的抽象 ATM 机. 只要一台机器实现了 AM 的功能(能计算, 能输出输入...), 程序都可以在上面运行, 不必关心这台机器是真实的, 还是用程序虚拟出来的。

用一句话来总结这三者的关系: **AM 在概念上定义了一台抽象计算机, 它从运行程序的视角刻画了一台计算机应该具备的功能, 而真机和 NEMU 都是这台抽象计算机的具体实现, 只是真机是通过物理上存在的数字电路来实现, NEMU 是通过程序来实现。**

如果你对面向对象程序设计有一些初步的了解, 解释起来就更简单了:

AM 是个抽象类, 真机和虚拟机是由 AM 这个抽象类派生出来的两个子类, 而 x86 真机和 NEMU 则分别是这两个子类的实例化。

AM 作为一个计算机的抽象模型, 可以将一个现代计算机从逻辑上划分成以下模块

AM = TRM + IOE + ASYE + PTE + MPE

- TRM(Turing Machine) - 图灵机, 为计算机提供基本的计算能力
- IOE(I/O Extension) - 输入输出扩展, 为计算机提供输出输入的能力
- ASYE(Asynchronous Extension) - 异步处理扩展, 为计算机提供处理中断异常的能力
- PTE(Protection Extension) - 保护扩展, 为计算机提供存储保护的能力
- MPE(Multi-Processor Extension) - 多处理器扩展, 为计算机提供多处理器通信的能力 (MPE 超出了 ICS 课程的范围, 在 PA 中不会涉及)

不同程序对计算机的功能需求也不完全一样, 例如只进行纯粹计算任务的程序在 TRM 上就可以运行; 要运行小游戏, 仅仅是 TRM 就不够了, 因为小游戏还需要和用户进行交互, 因此还需要 IOE; 要运行一个现代操作系统, 还要在此基础上加入 ASYE 和 PTE. 我们知道 ISA 是计算机系统软硬件接口, 而从上述 AM 的模块划分可以看出, AM 描述的恰恰就是 ISA 本身, 它是不同 ISA 的抽象. 感谢 AM 项目的诞生, 让 NEMU 和程序的界线更加泾渭分明, 同时使得 PA 的流程更加明确:

(在 NEMU 中)实现硬件功能 -> (在 AM 中)提供软件抽象 -> (在 APP 层)运行程序
(在 NEMU 中)实现更强大的硬件功能 -> (在 AM 中)提供更丰富的软件抽象 -> (在 APP 层)运行更复杂的程序

这个流程其实与 PA1 中开天辟地的故事遥相呼应: 先驱希望创造一个计算机的世界, 并赋予它执行程序的生命. 亲自搭建 NEMU(硬件)和 AM(软件)之间的桥梁来支撑程序的运行, 是"理解程序如何在计算机上运行"这一终极目标的不二选择。

RTFSC(3)

我们来简单介绍一下 AM 项目的代码. 代码中 nexus-am 目录下的源文件组织如下(部分目录下的文件并未列出):

```
nexus-am
├── am                                # AM相关
│   ├── am.h
│   ├── arch                        # 不同体系结构-平台的AM实现
│   │   ├── native
│   │   └── x86-nemu                # x86-nemu的AM实现
│   │       ├── img                  # 构建/运行二进制文件/镜像的脚本
│   │       │   ├── boot
│   │       │   │   └── Makefile
│   │       │   └── start.S          # 程序入口
│   │       ├── build                # 构建脚本
│   │       ├── loader.ld            # 链接脚本
│   │       └── run                   # 运行脚本
│   ├── include
│   ├── README.md
│   └── src
│       ├── asye.c                  # ASYE
│       ├── ioe.c                   # IOE
│       ├── pte.c                   # PTE
│       ├── trap.S
│       └── trm.c                   # TRM
└── Makefile
```

```

├─ apps                                # 直接运行在AM上的应用
├─ libs                                # 可以直接运行在AM上的库
├─ Makefile
├─ Makefile.app
├─ Makefile.check
├─ Makefile.compile
├─ Makefile.lib
├─ README.md
├─ SPEC.md
└─ tests                                # 直接运行在AM上的测试

```

整个 AM 项目分为三大部分：

- ❖ nexus-am/am - 不同计算机架构的 AM 实现, 在 PA 中我们只需要关注 nexus-am/am/arch/x86-nemu 即可
- ❖ nexus-am/tests 和 nexus-am/apps - 一些功能测试和直接运行 AM 上的应用程序
- ❖ nexus-am/libs - 一些体系结构无关的, 可以直接运行在 AM 上的库, 方便应用程序的开发

在让 NEMU 运行客户程序之前, 我们需要将客户程序的代码编译成可执行文件. 需要说明的是, 我们不能使用 gcc 的默认选项直接编译, 因为默认选项会根据 GNU/Linux 的运行环境将代码编译成运行在 GNU/Linux 下的可执行文件. 但此时的 NEMU 并不能为客户程序提供 GNU/Linux 的运行环境, 在 NEMU 中运行上述可执行文件会产生错误, 因此我们不能使用 gcc 的默认选项来编译用户程序.

解决这个问题的方法是交叉编译, 我们需要在 GNU/Linux 下根据 AM 的运行环境编译出能够在 NEMU 中运行的可执行文件. 为了不让链接器 ld 使用默认的方式链接, 我们还需要提供描述 AM 运行时环境的链接脚本. AM 的框架代码已经把相应的配置准备好了:

- ❖ gcc 将 AM 实现的源文件编译成目标文件, 然后通过 ar 将这些目标文件打包成一个归档文件作为一个库, 把不同计算机架构的 AM 实现通过库的方式提供给程序
- ❖ gcc 把在 AM 上运行的应用程序源文件编译成目标文件
- ❖ 必要的时候通过 gcc 和 ar 把程序依赖的运行库也打包成归档文件
- ❖ 执行脚本文件 nexus-am/am/arch/x86-nemu/img/build, 在脚本文件中
 - 将程序入口 nexus-am/am/arch/x86-nemu/img/boot/start.S 编译成目标文件
 - 最后让 ld 根据链接脚本 nexus-am/am/arch/x86-nemu/img/loader.ld, 将上述目标文件和归档文件链接成可执行文件

根据这一链接脚本的指示, 可执行程序重定位后的节从 0x100000 开始, 首先是 .text 节, 其中又以 nexus-am/am/arch/x86-nemu/img/boot/start.o 中自定义的 entry 节开始, 然后接下来是其它目标文件的 .text 节. 这样, 可执行程序 0x100000 处总是放置 nexus-am/am/arch/x86-nemu/img/boot/start.S 的代码, 而不是其它代码, 保证客户程序总能从 0x100000 开始正确执行. 链接脚本也定义了其它节 (包括 .rodata, .data, .bss) 的链接顺序, 还定义了一些关于位置信息的符号, 包括每个节的末尾, 栈顶位置, 堆区的起始和末尾.

我们对编译得到的可执行文件的行为进行简单的梳理:

- ✧ 第一条指令从 nexus-am/am/arch/x86-nemu/img/boot/start.S 开始, 设置好栈顶之后就跳转到 nexus-am/am/arch/x86-nemu/src/trm.c 的 _trm_init() 函数处执行.
- ✧ 在 _trm_init() 中调用 main() 函数执行程序的主体功能.
- ✧ 从 main() 函数返回后, 调用 _halt() 结束运行.

阅读 nexus-am/am/arch/x86-nemu/src/trm.c 中的代码, 你会发现只需要实现很少的 API 就可以支撑起程序在 TRM 上运行了:

- ✧ _Area _heap 结构用于指示堆区的起始和末尾
- ✧ void _putc(char ch) 用于输出一个字符
- ✧ void _halt(int code) 用于结束程序的运行
- ✧ void _trm_init() 用于进行 TRM 相关的初始化工作

这是因为, TRM 所需要的指令集和内存已经被编译器考虑进去了: 编译器认为, 硬件需要提供具体的指令集实现和可用的内存, 编译生成的程序里面只需要包含 "使用的指令" 和 "程序的内存映象" 这两方面的信息, 程序就可以在硬件上运行了, 所以我们不需要在 trm.c 里面提供 "使用指令集" 和 "使用内存" 的 API. 关于 AM 定义的 API, 可以阅读 nexus-am/README.md 和 nexus-am/SPEC.md.

堆和栈在哪里?

我们知道代码和数据都在可执行文件里面,但却没有提到堆(heap)和栈(stack).为什么堆和栈的内容没有放入可执行文件里面?那程序运行时刻用到的堆和栈又是怎么来的?AM的代码是否能给你带来一些启发?

把`_putc()`作为TRM的API是一个很有趣的考虑,我们在不久的将来再讨论它,目前我们暂不打算运行需要调用`_putc()`的程序.

最后来看看`_halt()`.`_halt()`里面是一条内联汇编语句,内联汇编语句允许我们在C代码中嵌入汇编语句.这条指令和我们常见的汇编指令不一样(例如`movl $1,%eax`),它是直接通过指令的编码给出的,它只有一个字节,就是`0xd6`.如果你在`nemu/src/cpu/exec/exec.c`中查看`opcode_table`,你会发现,这条指令正是那条特殊的`nemu_trap`!这其实也说明了为什么要通过编码来给出这条指令,如果你使用以下方式给出指令,汇编器将会报错:`asm volatile("nemu_trap" : : "a" (0))`

因为这条特殊的指令是我们人为添加的,标准的汇编器并不能识别它.如果你查看`objdump`的反汇编结果,你会看到`nemu_trap`指令被标识为(bad),原因是类似的:`objdump`并不能识别我们人为添加的`nemu_trap`指令."`a`"(0)表示在执行内联汇编语句给出的汇编代码之前,先将0读入`%eax`寄存器.这样,这段汇编代码的功能就和`nemu/src/cpu/exec/special.c`中的`helper`函数`nemu_trap()`对应起来了.此外,`volatile`是C语言的一个关键字,如果你想了解关于`volatile`的更多信息,请查阅相关资料.

运行更多的程序

未测试代码永远是错的,你需要足够多的测试用例来测试你的NEMU.我们在`nexus-am/tests/cputest`目录下准备了一些测试用例.首先我们让AM项目上的程序默认编译到`x86-nemu`的AM中:

```
--- nexus-am/Makefile.check
+++ nexus-am/Makefile.check
@@ -7,2 +7,2 @@
-ARCH ?= native
+ARCH ?= x86-nemu
ARCH = $(shell ls $(AM_HOME)/am/arch/)
```

然后在`nexus-am/tests/cputest/`目录下执行

```
make ALL=xxx run
```

其中`xxx`为测试用例的名称(不包含.c后缀).

实现更多的指令

你需要实现上文中提到的更多指令,以通过上述测试用例.

你可以自由选择按照什么顺序来实现指令.经过PA1的训练之后,你应该不会实现所有指令之后才进行测试了.要养成尽早做测试的好习惯,一般原则都是"实现尽可能少的指令来进行下一次的测试".你不需要实现所有指令的所有形式,只需要通过这些测试即可.如果将来仍然遇到了未实现的指令,就到时候再实现它们.

需要注意的是,`push imm8`指令需要对立即数进行符号扩展,这一点在i386手册中并没有明确说明.在IA-32手册中关于`push`指令有如下说明:

If the source operand is an immediate and its size is less than the operand size, a sign-extended value is pushed on the stack.

由于部分测试用例需要实现较多指令,建议按照以下顺序进行测试:

1. 其它
2. string
3. hello-str

基础设施(2)

测试与调试

理解指令的执行过程之后,添加各种指令更多的是工程实现.工程实现难免会碰到bug,实现不正确的时候如何快速进行调试,其实也属于基础设施的范畴.思考一下,译码查找表中有那么多指令,每一条指令又通过若干RTL指令实现,如果其中实现有误,我们该如何发现呢?

直觉上这貌似不是一件容易的事情,不过让我们来讨论一下其中的缘由.假设我们不小心把译码查找表中的某一条指令的译码函数填错了,NEMU执行到这一条指令的时候,就会使用错误的译码函数进行译码,从而导致执

行函数拿到了错误的源操作数,或者是将正确的结果写入了错误的目的操作数.这样,NEMU 执行这条指令的结果就违反了它原来的语义,接下来就会导致跟这条指令有依赖关系的其它指令也无法正确地执行.最终,我们就会看到客户程序访问内存越界,陷入死循环,或者 HIT BAD TRAP,甚至是 NEMU 触发了段错误.

调试的工具与原理

我们可以从上面的这个例子中抽象出一些软件工程相关的概念:

- ❖ **Fault**:实现错误的代码,例如填写了错误的译码函数
- ❖ **Error**:程序执行时不符合预期的状态,例如客户程序的指令没有被正确地执行
- ❖ **Failure**:能直接观测到的错误,例如 HIT BAD TRAP,段错误等

调试其实就是从观测到的 **failure** 一步一步回溯寻找 **fault** 的过程,找到了 **fault** 之后,我们就很快知道应该如何修改错误的代码了.但从上面的例子也可以看出,调试之所以不容易,恰恰是因为:

- ❖ **fault** 不一定马上触发 **error**
- ❖ 触发了 **error** 也不一定马上转变成可观测的 **failure**
- ❖ **error** 会像滚雪球一般越积越多,当我们观测到 **failure** 的时候,其实已经距离 **fault** 非常遥远了

理解了这些原因之后,我们就可以制定相应的策略了:

- ❖ 尽可能把 **fault** 转变成 **error**.这其实就是测试做的事情,所以 `nexus-am/tests/` 目录下提供了各种各样的测试用例.但并不是有了测试用例就能把所有 **fault** 都转变成 **error** 了,因为这取决于测试的覆盖度.要设计出一套全覆盖的测试并不是一件简单的事情,越是复杂的系统,全覆盖的测试就越难设计.至少,框架代码中提供的测试用例的覆盖度还是很有限的.但是,如何提高测试的覆盖度,是学术界一直以来都在关注的问题.
- ❖ 尽早观测到 **error** 的存在.观测到 **error** 的时机直接决定了调试的难度:如果等到触发 **failure** 的时候才发现 **error** 的存在,调试就会比较困难;但如果能在 **error** 刚刚触发的时候就观测到它,调试难度也就大大降低了.事实上,你已经见识过一些有用的工具了:
 - ✧ **-Wall, -Werror**:在编译时刻把潜在的 **fault** 直接转变成 **failure**.这种工具的作用很有限,只能寻找一些在编译时刻也觉得可疑的 **fault**,例如 `if(p=NULL)`,但也是代价最低的.
 - ✧ **assert()**:在运行时刻把 **error** 直接转变成 **failure**.`assert()` 是一个很简单却又非常强大的工具,只要在代码中定义好程序应该满足的特征,就一定能在运行时刻将不满足这些特征的 **error** 拦截下来.例如链表的实现,我们只需要在代码中插入一些很简单的 `assert()` (例如指针不为空),就能够几乎告别段错误.事实上,客户程序之所以会 HIT BAD TRAP,其实也是因为违背了我们设置的 `nemu_assert()`.但是,编写这些 `assert()` 其实需要我们对程序的行为有一定的了解,同时在程序特征不易表达的时候,`assert()` 的作用也较为有限.
 - ✧ **printf()**:通过输出的方式观察潜在的 **error**.这是用于回溯 **fault** 时最常用的工具,用于观测程序中的变量是否进入了错误的状态.在 NEMU 中我们提供了输出更多调试信息的宏 `Log()`,它实际上封装了 `printf()` 的功能.但由于 `printf()` 需要根据输出的结果人工判断是否正确,在便利程度上相对于 `assert()` 的自动判断就逊色了不少.
 - ✧ **GDB**:随时随地观测程序的任何状态.调试器是最强大的工具,但你需要在程序行为的茫茫大海中观测那些可疑的状态,因此使用起来的代价也是最大的.

根据上面的分析,我们就可以总结出一些调试的建议:

- ❖ 总是使用 **-Wall** 和 **-Werror**
- ❖ 尽可能多地在代码中插入 `assert()`
- ❖ `assert()` 无法捕捉到 **error** 时,通过 `printf()` 输出可疑的变量,期望能观测到 **error**
- ❖ `printf()` 不易观测 **error** 时,通过 GDB 理解程序的细致行为

Differential Testing

如果你在程序设计课上听说过上述这些建议,相信你几乎不会遇到过运行时错误.然而回过头来看上文提到的指令实现的 bug,我们会发现,这些工具还是不够用:我们很难通过 `assert()` 来表达指令的正确行为来进行自动检查,而 `printf()` 和 GDB 实际上并没有缩短 **error** 和 **failure** 的距离.

如果有一种方法能够表达指令的正确行为,我们就可以基于这种方法来进行类似 `assert()` 的检查了.那么,究竟什么地方表达了指令的正确行为呢?最直接的,当然就是 i386 手册了,但是我们恰恰就是根据 i386 手册中的指令行为来在 NEMU 中实现指令的,同一套方法不能既用于实现也用于检查.如果有一个 i386 手册的参考实现就好了.嘿!我们用的真机不就是根据 i386 手册实现出来的吗?我们让在 NEMU 中执行的每条指令也在真机中执行一

次,然后对比 NEMU 和真机的状态,如果 NEMU 和真机的状态不一致,我们就捕捉到 error 了!

这实际上是一种非常奏效的测试方法,在软件测试领域称为 **differential testing**. 我们刚才提到了"状态",那"状态"具体指的是什么呢?我们在 PA1 中已经认识到,计算机就是一个数字电路.那么,"计算机的状态"就恰恰是那些时序逻辑部件的状态,也就是寄存器和内存的值.其实仔细思考一下,计算机执行指令,就是修改这些时序逻辑部件的状态的过程.要检查指令的实现是否正确,只要检查这些时序逻辑部件中的值是否一致就可以了!Differential testing 可以非常及时地捕捉到 error,第一次发现 NEMU 的寄存器或内存的值与真机不一样的时候,就是因为当时执行的指令实现有误导致的.这时候其实离 error 非常接近,防止了 error 进一步传播的同时,要回溯找到 fault 也容易得多.

多么美妙的功能啊!背后还蕴含着计算机本质的深刻原理!但很遗憾,不要忘记了,真机上是运行了操作系统 GNU/Linux 的,而 NEMU 中的测试程序是运行在 AM 上的.就如前文所说 它们提供的运行时环境是不一样的,我们无法在 GNU/Linux 中运行基于 x86-nemu 的 AM 程序.所以,我们需要的不仅是一个 i386 手册的正确实现,而且需要要在上面能正确运行基于 x86-nemu 的 AM 程序.

事实上, QEMU 就是一个不错的参考实现.它是一个虚拟出来的完整的 x86 计算机系统,而 NEMU 的目标只是虚拟出 x86 的一个子集,能在 NEMU 上运行的程序,自然也能在 QEMU 上运行.因此,为了通过 differential testing 的方法测试 NEMU 实现的正确性,我们让 NEMU 和 QEMU 逐条指令地执行同一个客户程序.双方每执行完一条指令,就检查各自的寄存器和内存的状态,如果发现状态不一致,就马上报告错误,停止客户程序的执行.

NEMU 的框架代码已经准备好相应的功能了,在 nemu/include/common.h 中定义宏 DIFF_TEST 之后,重新编译 NEMU 后运行,你会发现 NEMU 多输出了 Connect to QEMU successfully 的信息.定义了宏 DIFF_TEST 之后,monitor 会多进行以下初始化工作,你不需要了解这些工作的具体细节,只需要知道这是为了让 QEMU 进入一个和 NEMU 同等的状态就可以了.

- ❖ 调用 init_difftest() 函数(在 nemu/src/monitor/diff-test/diff-test.c 中定义)来启动 QEMU.需要注意的是,框架代码让 QEMU 运行在后台,因此你将看不到 QEMU 的任何输出.
- ❖ 在 load_img() 的最后将客户程序拷贝一份副本到 QEMU 模拟的内存中.
- ❖ 在 restart() 中调用 init_qemu_reg() 函数(在 nemu/src/monitor/diff-test/diff-test.c 中定义),来把 QEMU 的通用寄存器设置成和 NEMU 一样.

进行了上述初始化工作之后, QEMU 和 NEMU 就处于相同的状态了.接下来就要进行逐条指令执行后的状态对比了,实现这一功能的是 difftest_step() 函数(在 nemu/src/monitor/diff-test/diff-test.c 中定义).它会在 exec_wrapper() 的最后被调用,在 NEMU 中执行完一条指令后,就在 difftest_step() 中让 QEMU 执行相同的指令,然后读出 QEMU 中的寄存器.你需要添加相应的代码,把 NEMU 的 8 个通用寄存器和 eip 与从 QEMU 中读出的寄存器的值进行比较,如果发现值不一样,就输出相应的提示信息,并将 diff 标志设置为 true.在 difftest_step() 的最后,如果检测到 diff 标志为 true,就停止客户程序的运行.

实现 differential testing

在 difftest_step() 中添加相应的代码,实现 differential testing 的核心功能.实现正确后,你将会得到一款无比强大的测试工具.

噢?我们不需要对内存的状态进行比较吗?事实上, NEMU 是通过一套 GDB 协议与 QEMU 通信来获取 QEMU 的状态的,但是通过这一协议还是不好获取指令修改的内存位置,而对比整个内存又会带来很大的开销,所以我们就不对内存的状态进行比较了.事实上, NEMU 中的简化实现也会导致某些寄存器的状态与 QEMU 的结果不一致,例如 EFLAGS, NEMU 只实现了 EFLAGS 中的少量标志位,同时也简化了某些指令对 EFLAGS 的更新.另外,一些特殊的系统寄存器也没有完整实现.因此,我们实现的 differential testing 并不是完整地对比 QEMU 和 NEMU 的状态,但是不管是内存还是标志位,只要客户程序的一条指令修改了它们,在不久的将来肯定也会再次用到它们,到时候一样能检测出状态的不同.同时框架中也准备了 is_skip_nemu 和 is_skip_qemu 这两个变量,用于跳过少量不易进行对比的指令.因此,我们其实牺牲了一些比较的精度,来换取性能的提升,但即使这样,由于 differential testing 需要与 QEMU 进行通信,这还是会吧 NEMU 的运行速度拉低上百倍.因此除非是在进行调试,否则不建议打开 differential testing 的功能来运行 NEMU.

一键回归测试

在实现指令的过程中,你需要逐个测试用例地运行.但在指令实现正确之后,是不是意味着可以和这些测试用例说再见呢?显然不是.以后你还需要在 NEMU 中加入新的功能,为了保证加入的新功能没有影响到已有功能的实现,你还需要重新运行这些测试用例.在软件测试中,这个过程称为回归测试.

将来还要重复运行这些测试用例,手动重新运行每一个测试是一种效率低下的做法.为了提高效率,我们提供了一个用于一键回归测试的脚本.在 nemu/ 目录下运行 `bash runall.sh` 来批量运行 nexus-am/tests/cputest/ 中的

所有测试,并报告每个测试用例的运行结果.如果一个测试用例运行失败,脚本将会保留相应的日志文件;当使用脚本通过这个测试用例的时候,日志文件将会被移除.

NEMU 的本质

你已经知道,NEMU 是一个用来执行其它程序的程序.在可计算理论中,这种程序有一个专门的名词,叫通用程序(Universal Program),它的通俗含义是:其它程序能做的事情,它也能做.通用程序的存在性有专门的证明,我们在这里不做深究,但是,我们可以写出 NEMU,可以用 Docker/虚拟机做实验,乃至我们可以在计算机上做各种各样的事情,其背后都蕴含着通用程序的思想:NEMU 和各种模拟器只不过是通用程序的实例化,我们也可以毫不夸张地说,计算机就是一个通用程序的实体化.通用程序的存在性为计算机的出现奠定了理论基础,是可计算理论中一个极其重要的结论,如果通用程序的存在性得不到证明,我们就没办法放心地使用计算机,同时也不能义正辞严地说"机器永远是对的".

我们编写的 NEMU 最终会被编译成 x86 机器代码,用 x86 指令来模拟 x86 程序的执行.事实上在 30 多年前(1983 年),Martin Davis 教授就在他出版的"Computability, complexity, and languages: fundamentals of theoretical computer science"一书中提出了一种仅有三种指令的程序设计语言 L 语言,并且证明了 L 语言和其它所有编程语言的计算能力等价.L 语言中的三种指令分别是:

```
V = V + 1
V = V - 1
IF V != 0 GOTO LABEL
```

用 x86 指令来描述,就 inc, dec 和 jne 三条指令.假设除了输入变量之外,其它变量的初值都是 0,并且假设程序执行到最后一条指令就结束,你可以仅用这三种指令写一个计算两个正整数相加的程序吗?

```
# Assume a = 0, x and y are initialized with some positive integers.
# Other temporary variables are initialized with 0.
# Let "jne" carries a variable: jne v, label.
# It means "jump to label if v != 0".
# Compute a = x + y used only these three instructions: inc, dec, jnz.
# No other instructions can be used.
# The result should be stored in variable "a".
# Have a try?
```

人更惊讶的是,Martin Davis 教授还证明了,在不考虑物理限制的情况下(认为内存容量无限多,每一个内存单元都可以存放任意大的数),用 L 语言也可以编写出一个和 NEMU 类似的通用程序!而且这个用 L 语言编写的通用程序的框架,竟然还和 NEMU 中的 cpu_exec() 函数如出一辙:取指,译码,执行...这其实并不是巧合,而是模拟(Simulation)在计算机科学中的应用.

早在 Martin Davis 教授提出 L 语言之前,科学家们就已经在探索什么问题是可以计算的了.回溯到 19 世纪 30 年代,为了试图回答这个问题,不同的科学家提出并研究了不同的计算模型,包括 Gödel, Herbrand 和 Kleen 研究的递归函数,Church 提出的 λ -演算,Turing 提出的图灵机,后来发现这些模型在计算能力上都是等价的;到了 40 年代,计算机就被制造出来了.后来甚至还有人证明了,如果使用无穷多个算盘拼接起来进行计算,其计算能力和图灵机等价!我们可以从中得出一个推论,通用程序在不同的计算模型中有不同的表现形式.NEMU 作为一个通用程序,在 19 世纪 30 年代有着非凡的意义.如果你能在 80 年前设计出 NEMU,说不定"图灵奖"就要用你的名字来命名了.计算的极限这一篇科普文章叙述了可计算理论的发展过程,我们强烈建议你阅读它,体会人类的文明(当然一些数学功底还是需要的).如果你对可计算理论感兴趣,可以选修宋方敏老师的计算理论导引课程.

把思绪回归到 PA 中,通用程序的性质告诉我们,NEMU 的潜力是无穷的.为了创造出一个缤纷多彩的世界,你觉得 NEMU 还缺少些什么呢?

捕捉死循环(有点难度)

NEMU 除了作为模拟器之外,还具有简单的调试功能,可以设置断点,查看程序状态.如果让你为 NEMU 添加如下功能

当用户程序陷入死循环时,让用户程序暂停下来,并输出相应的提示信息

你觉得应该如何实现?如果你感到疑惑,在互联网上搜索相关信息:

温馨提示

PA2 阶段 2 到此结束.

输入输出

我们已经成功运行了各个 `cputest` 中的测试用例,但这些测试用例都只能默默地进行纯粹的计算.回想起我们在程序设计课上写的第一个程序 `hello`,至少也输出了一句话.事实上,输入输出是计算机与外界交互的基本手段,如果你还记得计算机刚启动时执行的 BIOS 程序的全称是 **Basic Input/Output System**,你就会理解输入输出对计算机来说是多么重要了.在真实的计算机中,输入输出都是通过 I/O 设备来完成的.

设备的工作原理其实没什么神秘的.你会在不久的将来在数字电路实验中看到键盘模块和 VGA 模块相关的 `verilog` 代码.噢,原来这些设备也一样是个数字电路!事实上,只要向设备发送一些有意义的数字信号,设备就会按照这些信号的含义来工作.让一些信号来指导设备如何工作,这不就像"程序的指令指导 CPU 如何工作"一样吗?恰恰就是这样!设备也有自己的状态寄存器(相当于 CPU 的寄存器),也有自己的功能部件(相当于 CPU 的运算器).当然不同的设备有不同的功能部件,例如键盘有一个把按键的模拟信号转换成扫描码的部件,而 VGA 则有一个把像素颜色信息转换成显示器模拟信号的部件.这些控制设备工作的信号称为"命令字",可以理解成"设备的指令",设备的工作就是负责接收命令字,并进行译码和执行...你已经知道 CPU 的工作方式,这一切对你来说都太熟悉了.唯一让你觉得神秘的,就要数设备功能部件中的模/数转换,数/模转换等各种有趣的实现.遗憾的是,我们的课程并没有为我们提供实践的机会,因此它们成为了一种神秘的存在.

我们希望计算机能够控制设备,让设备做我们想要做的事情,这一重任毫无悬念地落到了 CPU 身上. CPU 除了进行运算之外,还需要与设备协作来完成不同的任务.要控制设备工作,就需要向设备发送命令字.接下来的问题是, CPU 怎么区分不同的设备?具体要怎么向一个设备发送命令字?

对第一个问题的回答涉及到 I/O 的编址方式.我们知道内存有地址的概念,类似地,我们也可以给 I/O 设备中允许 CPU 访问的寄存器逐一编址. I/O 编址的目的就是让 CPU 可以区分不同的设备,尽管这种区分的方式在我们来看非常笨拙:只是让不同的设备报个数而已.

一种 I/O 编址方式是端口映射 I/O (port-mapped I/O), CPU 使用专门的 I/O 指令对设备进行访问,并把设备的地址称作端口号.有了端口号以后,在 I/O 指令中给出端口号,就知道要访问哪一个设备的哪一个寄存器了.市场上的计算机绝大多数都是 IBM PC 兼容机, IBM PC 兼容机对常见设备端口号的分配有专门的规定.设备中可能会有一些私有寄存器,它们是由设备自己维护的,它们没有端口号, CPU 不能直接访问它们.

x86 提供了 `in` 和 `out` 指令用于访问设备,其中 `in` 指令用于将设备寄存器中的数据传输到 CPU 寄存器中, `out` 指令用于将 CPU 寄存器中的数据传送到设备寄存器中. 一个例子是 `nexus-am/am/arch/x86-nemu/src/trm.c` 中 `serial_init()` 的代码,代码使用 `out` 指令给串口发送命令字.例如

```
movl $0x0, %al
movl $0x3f9, %edx
outb %al, (%dx)
```

上述代码把数据 `0x0` 传送到 `0x3f9` 号端口所对应的设备寄存器中. 你要注意区分 I/O 指令和命令字, I/O 指令是 CPU 执行的,作用是对设备寄存器进行读写;而命令字是设备来执行的,作用和设备相关,由设备来解释和执行. CPU 执行上述代码后,会将 `0x0` 这个数据传送到串口的一个寄存器中,串口接收到 `0x0` 后,把它解释成一条命令,发现是一条关中断命令,于是就会进入关中断状态;但对 CPU 来说,它并不关心 `0x0` 的含义,只会老老实实地把 `0x0` 传送到 `0x3f9` 号端口.至于设备接收到 `0x0` 之后会做什么,那就是设备自己的事情了.事实上,设备的行为都会在相应的文档里面有清晰的定义,驱动开发者需要阅读设备的相关文档,编写相应的命令字序列来对设备进行期望的操作.在 PA 中我们无需了解这些细节,只需要知道,我们可以通过阅读相关文档,编写相应的程序在 CPU 上运行来操作设备即可.

端口映射 I/O 把端口号作为 I/O 指令的一部分,这种方法很简单,但同时也是它最大的缺点.指令集为了兼容已经开发的程序,是只能添加但不能修改的.这意味着,端口映射 I/O 所能访问的 I/O 地址空间的大小,在设计 I/O 指令的那一刻就已经决定下来了.所谓 I/O 地址空间,其实就是所有能访问的设备的地址的集合.随着设备越来越多,功能也越来越复杂, I/O 地址空间有限的端口映射 I/O 已经逐渐不能满足需求了.有的设备需要让 CPU 访问一段较大的连续存储空间,如 VGA 的显存, 24 色加上 Alpha 通道的 1024x768 分辨率的显存就需要 3MB 的编址范围.于是内存映射 I/O (memory-mapped I/O) 应运而生.

内存映射 I/O 这种编址方式非常巧妙,它是通过不同的物理内存地址给设备编址的.这种编址方式将一部分物理内存"重定向"到 I/O 地址空间中, CPU 尝试访问这部分物理内存的时候,实际上最终是访问了相应的 I/O 设备, CPU 却浑然不知.这样以后, CPU 就可以通过普通的访存指令来访问设备.这也是内存映射 I/O 得天独厚的长处:物理内存的地址空间和 CPU 的位宽都会不断增长,内存映射 I/O 从来不需要担心 I/O 地址空间耗尽的问题.从原理上来说,内存映射 I/O 唯一的缺点就是, CPU 无法通过正常渠道直接访问那些被映射到 I/O 地址空间的物理内存了.但随着计算机的发展,内存映射 I/O 的唯一缺点已经越来越不明显了:现代计算机都已经是 64 位计算机,

物理地址线都有 48 根,这意味着物理地址空间有 256TB 这么大,从里面划出 3MB 的地址空间给显存,根本就是不痛不痒.正因为如此,内存映射 I/O 成为了现代计算机主流的 I/O 编址方式:RISC 架构只提供内存映射 I/O 的编址方式,而 PCI-e,网卡,x86 的 APIC 等主流设备,都支持通过内存映射 I/O 来访问.

内存映射 I/O 的一个例子是 NEMU 中的物理地址区间 [0x40000, 0x80000). 这段物理地址区间被映射到 VGA 内部的显存,读写这段物理地址区间就相当于对读写 VGA 显存的数据.例如

```
memset((void *)0x40000, 0, SCR_SIZE);
```

会将显存中一个屏幕大小的数据清零,即往整个屏幕写入黑色像素,作用相当于清屏.可以看到,内存映射 I/O 的编程模型和普通的编程完全一样:程序员可以直接把 I/O 设备当做内存来访问.这一特性也是深受驱动开发者的喜爱.

理解 volatile 关键字

也许你从来都没听说过 C 语言中有 volatile 这个关键字,但它从 C 语言诞生开始就一直存在. volatile 关键字的作用十分特别,它的作用是避免编译器对相应代码进行优化.你应该动手体会一下 volatile 的作用,在 GNU/Linux 下编写以下代码:

```
void fun() {
    volatile unsigned char *p = (void *)0x8049000;
    *p = 0;
    while(*p != 0xff);
    *p = 0x33;
    *p = 0x34;
    *p = 0x86;
}
```

然后使用 -O2 编译代码.尝试去掉代码中的 volatile 关键字,重新使用 -O2 编译,并对比去掉 volatile 前后反汇编结果的不同.

你或许会感到疑惑,代码优化不是一件好事情吗?为什么会有 volatile 这种奇葩的存在?思考一下,如果代码中的地址 0x8049000 最终被映射到一个设备寄存器,去掉 volatile 可能会带来什么问题?

加入 IOE

NEMU 框架代码中已经提供了设备的代码,位于 nemu/src/device 目录下.代码提供了以下模块的模拟:

- ❖ 端口映射 I/O 和内存映射 I/O 两种 I/O 编址方式
- ❖ 串口,时钟,键盘,VGA 四种设备

为了简化实现,所有设备都是不可编程的,只实现了在 NEMU 中用到的功能.我们对代码稍作解释.

- ❖ nemu/src/device/io/port-io.c 是对端口 I/O 的模拟.其中 PIO_t 结构用于记录一个端口 I/O 映射的关系,设备会初始化时会调用 add_pio_map() 函数来注册一个端口 I/O 映射关系,返回该映射关系的 I/O 空间首地址.pio_read()和 pio_write()是面向 CPU 的端口 I/O 读写接口.由于 NEMU 是单线程程序,因此只能串行模拟整个计算机系统的工作,每次进行 I/O 读写的时候,才会调用设备提供的回调函数(callback),更新设备的状态.内存映射 I/O 的模拟和端口 I/O 的模拟比较相似,只是内存映射 I/O 的读写并不是面向 CPU 的,这一点会在下文进行说明.
- ❖ nemu/src/device/device.c 含有和 SDL 库相关的代码,NEMU 使用 SDL 库来模拟计算机的标准输入输出.init_device()函数首先对以上四个设备进行初始化,其中在初始化 VGA 时还会进行一些和 SDL 相关的初始化工作,包括创建窗口,设置显示模式等.最后还会注册一个 100Hz 的定时器,每隔 0.01 秒就会调用一次 device_update()函数.device_update()函数主要进行一些设备的模拟操作,包括以 50Hz 的频率刷新屏幕,以及检测是否有按键按下/释放.需要说明的是,代码中注册的定时器是虚拟定时器,它只会在 NEMU 处于用户态的时候进行计时:如果 NEMU 在 ui_mainloop()中等待用户输入,定时器将不会计时;如果 NEMU 进行大量的输出,定时器的计时将会变得缓慢.因此除非你在进行调试,否则尽量避免大量输出的情况,从而影响定时器的.

提供的代码是模块化的,要在 NEMU 中加入 IOE,你只需要在原来的代码上作少量改动:在 nemu/include/common.h 中定义宏 HAS_IOE.定义后,init_device()函数会对设备进行初始化.重新编译后,你会看到运行 NEMU 时会弹出一个新窗口,用于显示 VGA 的输出(见下文).另一方面,我们还需要在 AM 中实现相应的 API 为程序提供 IOE 的抽象(在 nexus-am/am/arch/x86-nemu/src/ioe.c 中定义):

- ❖ unsigned long _uptime() 用于返回系统启动后经过的毫秒数

- ❖ `int _read_key()` 用于返回按键的键盘码,若无按键,则返回 `_KEY_NONE`
- ❖ `_Screen _screen` 结构用于指示屏幕的大小
- ❖ `void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h)` 用于将 `pixels` 指定的矩形像素绘制到屏幕中以 `(x, y)` 和 `(x+w, y+h)` 两点连线为对角线的矩形区域
- ❖ `void _draw_sync()` 用于将之前的绘制内容同步到屏幕上(在 `NEMU` 中绘制内容总是会同步到屏幕上,因而无需实现此 API)
- ❖ `void _ioe_init()` 用于进行 IOE 相关的初始化工作,调用后程序才能正确使用上述 IOE 相关的 API

下面我们来逐一介绍如何在 `AM` 中添加 IOE 的功能来支撑程序的运行。

串口

串口是最简单的输出设备。`nemu/src/device/serial.c` 模拟了串口的功能。其大部分功能也被简化,只保留了数据寄存器和状态寄存器。串口初始化时会注册 `0x3F8` 处长度为 8 个字节的端口作为其寄存器,但代码中只模拟了其中的两个寄存器的功能。由于 `NEMU` 串行模拟计算机系统的工作,串口的状态寄存器可以一直处于空闲状态;每当 CPU 往数据寄存器中写入数据时,串口会将数据传送到主机的标准输出。

事实上,我们之前提到的 `_putc()` 函数,就是通过串口输出的。然而 `AM` 却把 `_putc()` 放在 `TRM`,而不是 `IOE` 中,这让人觉得有点奇怪。的确,可计算理论中提出的最原始的 `TRM` 并不包含输出的能力,但对于一个现实的计算机系统来说,输出是一个最基本的功能,没有输出,用户甚至无法知道程序具体在做什么。因此在 `AM` 中, `_putc()` 的加入让 `TRM` 具有输出字符的能力,被扩充后的 `TRM` 更接近一个实用的机器,而不再是只会计算的数学模型。

`nexus-am/am/arch/x86-nemu/src/trm.c` 中已经提供了串口的功能。为了让程序使用串口进行输出,你还需要在 `NEMU` 中实现端口映射 I/O。

运行 Hello World

实现 `in, out` 指令,在它们的 `helper` 函数中分别调用 `pio_read()` 和 `pio_write()` 函数。由于 `NEMU` 中有一些设备的行为是我们自定义的,与 `QEMU` 中的标准设备的行为不完全一样(例如 `NEMU` 中的串口总是就绪的,但 `QEMU` 中的串口并不是这样),这导致在 `NEMU` 中执行 `in` 和 `out` 指令的结果与 `QEMU` 可能会存在不可调整的偏差。为了使得 `differential testing` 可以正常工作,我们在这两条指令中调用了相应的函数来设置 `is_skip_qemu` 标志,来跳过与 `QEMU` 的检查。

实现后,在 `nexus-am/am/arch/x86-nemu/src/trm.c` 中定义宏 `HAS_SERIAL`,然后在 `nexus-am/apps/hello` 目录下键入 `make run`,在 `NEMU` 中运行基于 `AM` 的 `hello` 程序。如果你的实现正确,你将会看到程序往终端输出了 10 行 `Hello World!`

需要注意的是,这个 `hello` 程序和我们在程序设计课上写的第一个 `hello` 程序所处的层次是不一样的:这个 `hello` 程序可以说是直接运行在裸机上,可以在 `AM` 的抽象下直接输出到设备(串口);而我们在程序设计课上写的 `hello` 程序位于操作系统之上,不能直接操作设备,只能通过操作系统提供的服务进行输出,输出的数据要经过很多层抽象才能到达设备层。我们会在 `PA3` 中进一步体会操作系统的作用。

时钟

有了时钟,程序才可以提供时间相关的体验,例如游戏的帧率,程序的快慢等。`nemu/src/device/timer.c` 模拟了 `i8253` 计时器的功能。计时器的大部分功能都被简化,只保留了“发起时钟中断”的功能(目前我们不会用到)。同时添加了一个自定义的 `RTC(Real Time Clock)`,初始化时将会注册 `0x48` 处的端口作为 `RTC` 寄存器,CPU 可以通过 I/O 指令访问这一寄存器,获得当前时间(单位是 `ms`)。

实现 IOE

实现 `_uptime()` 后,在 `NEMU` 中运行 `timetest` 程序(在 `nexus-am/tests/timetest` 目录下,编译和运行方式请参考上文,此后不再额外说明)。如果你的实现正确,你将会看到程序每隔 1 秒输出一句话。

native 作为 AM

“`native`”是指操作系统默认的运行时环境,例如我们通过 `gcc hello.c` 编译程序时,就会编译到 `GNU/Linux` 提供的运行时环境。事实上,`native` 也可以看做一个简单的 `AM`,目前只支持 `TRM` 和 `IOE`。但很快你就会看到,`native` 也已经可以支撑很多程序的运行了。

看看 NEMU 跑多快

有了时钟之后,我们就可以测试一个程序跑多快,从而测试计算机的性能。尝试在 `NEMU` 中依次运行以下 `benchmark`(已经按照程序的复杂度排序,均在 `nexus-am/apps` 目录下;另外跑分时请注释掉 `nemu/include/common.h` 中的 `DEBUG` 和 `DIFF_TEST` 宏,以获得较为真实的跑分):

- ❖ Dhrystone
- ❖ Coremark
- ❖ microbench

成功运行后会输出跑分. 跑分以 i7-6700 @ 3.40GHz 的处理器为参照, 100000 分表示与参照机器性能相当, 100 分表示性能为参照机器的千分之一. 除了和参照机器比较之外, 也可以和小伙伴进行比较. 如果把上述 benchmark 编译到 native (编译和运行时添加 ARCH=native 参数), 还可以比较 native 的性能.

另外, microbench 提供了两个不同规模的测试集 test 和 ref. 其中 ref 测试集规模较大, 用于跑分测试, 默认会编译 ref 测试集; test 测试集规模较小, 用于正确性测试, 需要在运行 make 时显式指定编译 test 测试集: `make INPUT=TEST`

键盘

键盘是最基本的输入设备. 一般键盘的工作方式如下: 当按下一个键的时候, 键盘将会发送该键的通码 (make code); 当释放一个键的时候, 键盘将会发送该键的断码 (break code). nemu/src/device/keyboard.c 模拟 i8042 通用设备接口芯片的功能. 其大部分功能也被简化, 只保留了键盘接口. i8042 初始化时会注册 0x60 处的端口作为数据寄存器, 注册 0x64 处的端口作为状态寄存器. 每当用户敲下/释放按键时, 将会把相应的键盘码放入数据寄存器, 同时把状态寄存器的标志设置为 1, 表示有按键事件发生. CPU 可以通过端口 I/O 访问这些寄存器, 获得键盘码. 在 AM 中, 我们约定通码的值为断码+0x8000.

如何检测多个键同时被按下

在游戏中, 很多时候需要判断玩家是否同时按下了多个键, 例如 RPG 游戏中的八方向行走, 格斗游戏中的组合招式等等. 根据键盘码的特性, 你知道这些功能是如何实现的吗?

实现 IOE (2)

实现 `_read_key()` 后, 在 NEMU 中运行 keytest 程序 (在 nexus-am/tests/keytest 目录下). 如果你的实现正确, 在程序运行时弹出的新窗口中按下按键, 你将会看到程序输出相应的按键信息.

VGA

VGA 可以用于显示颜色像素, 是最常用的输出设备. nemu/src/device/vga.c 模拟了 VGA 的功能. VGA 初始化时注册了从 0x40000 开始的一段用于映射到 video memory 的物理内存. 在 NEMU 中, video memory 是唯一使用内存映射 I/O 方式访问的 I/O 空间. 代码只模拟了 400x300x32 的图形模式, 一个像素占 32 个 bit 的存储空间, R(red), G(green), B(blue), A(alpha) 各占 8 bit, 其中 VGA 不使用 alpha 的信息. 如果你对 VGA 编程感兴趣, 这里有一个名为 FreeVGA 的项目, 里面提供了很多 VGA 的相关资料.

神奇的调色板

现代的显示器一般都支持 24 位的颜色 (R, G, B 各占 8 个 bit, 共有 $2^8 \times 2^8 \times 2^8$ 约 1600 万种颜色), 为了让屏幕显示不同的颜色成为可能, 在 8 位颜色深度时会使用调色板的概念. 调色板是一个颜色信息的数组, 每一个元素占 4 个字节, 分别代表 R(red), G(green), B(blue), A(alpha) 的值. 引入了调色板的概念之后, 一个像素存储的就不再是颜色的信息, 而是一个调色板的索引: 具体来说, 要得到一个像素的颜色信息, 就要把它的值当作下标, 在调色板这个数组中做下标运算, 取出相应的颜色信息. 因此, 只要使用不同的调色板, 就可以在不同的时刻使用不同的 256 种颜色了.

在一些 90 年代的游戏里, 很多渐出渐入效果都是通过调色板实现的, 聪明的你知道其中的玄机吗?

添加内存映射 I/O

在 `paddr_read()` 和 `paddr_write()` 中加入对内存映射 I/O 的判断. 通过 `is_mmio()` 函数判断一个物理地址是否被映射到 I/O 空间, 如果是, `is_mmio()` 会返回映射号, 否则返回 -1. 内存映射 I/O 的访问需要调用 `mmio_read()` 或 `mmio_write()`, 调用时需要提供映射号. 如果不是内存映射 I/O 的访问, 就访问 pmem.

实现后, 在 NEMU 中运行 videotest 程序 (在 nexus-am/tests/videotest 目录下). 如果内存映射 I/O 实现正确, 你会看到新窗口中输出了一些颜色信息.

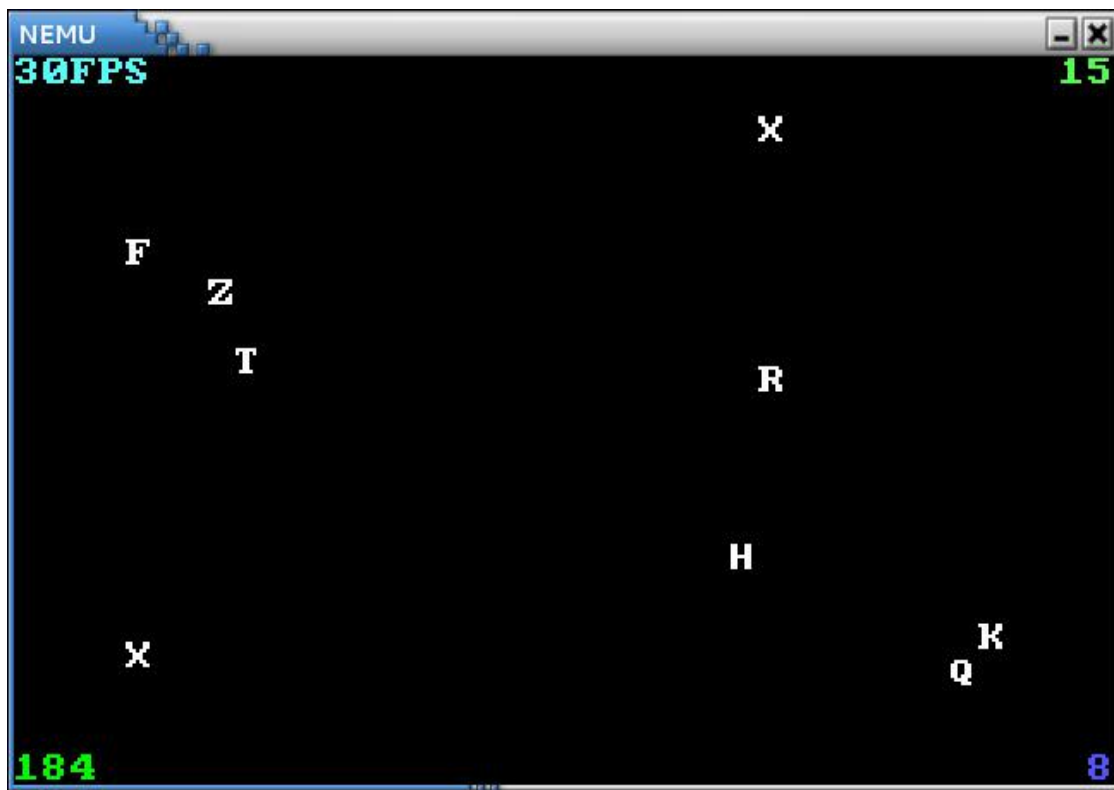
实现 IOE (3)

事实上, 刚才输出的颜色信息并不是 videotest 输出的画面, 这是因为框架代码中的 `_draw_rect()` 并未正确实现其功能. 你需要实现正确的 `_draw_rect()`. 实现后, 在 NEMU 中重新运行 videotest. 如果你的实现正确, 你将会看到新窗口中输出了相应的动画效果.

运行打字小游戏

在 NEMU 和 AM 中都完整实现 IOE 后, 我们就可以运行打字小游戏了 (在 nexus-am/apps/typing 目录下). 打字小游戏来源于 2013 年 NJUCS oslab0 的框架代码. 为了配合移植, 代码的结构做了少量调整, 同时去掉了和显存

优化相关的部分,并去掉了浮点数.



有兴趣折腾的同学可以尝试在 NEMU 中运行 `litenes` (在 `nexus-am/apps/litenes` 目录下). 没错,我们在 PA1 的开头给大家介绍的红白机模拟器,现在也已经可以在 NEMU 中运行起来了!

事实上,我们已经实现了一个冯诺依曼计算机系统!你已经在导论课上学习到,冯诺依曼计算机系统由 5 个部件组成:运算器,控制器,存储器,输入设备和输出设备.何况这些咋听之下让人云里雾里的名词,现在都已经跃然"码"上:你已经在 NEMU 中把它们都实现了!再回过头来审视这一既简单又复杂的计算机系统:说它简单,它只不过在 TRM 的基础上添加了 IOE,本质上还是"取指->译码->执行"的工作方式,甚至只要具备一些数字电路的知识就可以理解构建计算机的可能性;说它复杂,它却已经足够强大来支撑这么多酷炫的程序,实在是让人激动不已啊!那些看似简单但又可以折射出无限可能的事物,其中承载的美妙规律容易使人们为之陶醉,为之折服.计算机,就是其中之一.

必答题

你需要在实验报告中用自己的语言,尽可能详细地回答下列问题.

- ❖ 在 `nemu/include/cpu/rtl.h` 中,你会看到由 `static inline` 开头定义的各种 RTL 指令函数.选择其中一个函数,分别尝试去掉 `static`,去掉 `inline` 或去掉两者,然后重新进行编译,你会看到发生错误.请分别解释为什么会发生这些错误?你有办法证明你的想法吗?
- ❖ 了解 Makefile 请描述你在 `nemu` 目录下敲入 `make` 后,`make` 程序如何组织 `.c` 和 `.h` 文件,最终生成可执行文件 `nemu/build/nemu`. (这个问题包括两个方面:Makefile 的工作方式和编译链接的过程.)关于 Makefile 工作方式的提示:
 - ✧ Makefile 中使用了变量,包含文件等特性
 - ✧ Makefile 运用并重写了一些 `implicit rules`
 - ✧ 在 `man make` 中搜索 `-n` 选项,也许会对你有帮助
 - ✧ RTFM

温馨提示

PA2 到此结束